

Lab 3: DDP / DALI Implementation & Profiling

CSED490F Deep Learning Implementation — Team 4

20262756 Kwangmo Yang (양광모), 20262346 Chanyoung Gwak (곽찬영)

1 구현

수정한 파일은 handler/와 scripts/launch.sh뿐이고, train_cifar.py는 건드리지 않았다. 모든 방식에서 전체 batch 크기를 512(train/test)로 유지해서, DP·DDP·DALI가 iteration마다 같은 양의 일을 한다.

P0 (Nsight 명령). scaffold의 학습 명령 앞에 nsys profile을 붙였다. 코드의 nvtx.annotate 구간을 기록하려면 NVTX trace가 필요하다. Vast.ai 컨테이너는 권한이 없어 CPU sampling은 켜지 않았다. mp.spawn으로 생긴 자식 프로세스도 같은 보고서에 기록된다.

```
CUDA_VISIBLE_DEVICES=$LOCAL_GPU_IDS nsys profile --trace=cuda,nvtx,osrt,cudnn,cublas \
--cuda-memory-usage=true --sample=none --cuptxsw=none --force-override=true \
--output="$NSIGHT_LOG_DIR/$NSIGHT_FILE_NAME" $NSYS_EXTRA_ARGS python train_cifar.py ...
```

P1-P3 (process group). run_process는 mp.spawn(func, args=(args,), nprocs=num_gpu)를 호출해 main_func(proc_id, args)를 GPU마다 한 번씩 실행한다. initialize_group은 먼저 torch.cuda.set_device(proc_id)를 호출하고 (CUDA_VISIBLE_DEVICES로 GPU를 제한하므로 rank가 곧 로컬 GPU 번호다), 그다음 dist.init_process_group("nccl", init_method="tcp://host:port", world_size, rank)를 호출한다. device를 먼저 지정해야 각 NCCL communicator가 자기 GPU에 묶인다. destroy_process는 finally 블록에서 불리므로, group이 초기화된 경우에만 dist.destroy_process_group()을 호출한다.

P4 (모델). DP는 프로세스 하나가 모든 GPU를 다루지만(device_ids=[0..N-1]), DDP는 프로세스마다 GPU를 하나씩 맡는다. dev = torch.cuda.current_device()에 대해 model.cuda(dev) 후 DDP(model, device_ids=[dev], output_device=dev)로 감싼다.

P5 (DDP 로더). train/test마다 DistributedSampler(num_replicas=world_size, rank=rank)를 만들고(train은 shuffle, test는 하지 않음), DataLoader에 sampler=로 넘기면서 shuffle 인자는 뺐다. 프로세스당 batch는 batch // world_size(나누어떨어지는지 assert)라서 rank마다 겹치지 않는 부분을 보고, 전체 batch는 DP와 같다. DDP/train.py가 지표를 all-reduce하므로 기록되는 loss와 정확도는 전체 기준이다.

P6 (DALI 파이프라인). torchvision 변환을 연산자 단위로 그대로 옮겼다.

torchvision (DP/DDP)	DALI (CifarPipeline.define_graph)
ImageFolder 읽기, shuffle	fn.readers.file(file_root, shard_id=rank, num_shards=N, random_shuffle=is_train, name="Reader")
PIL 디코딩	fn.decoders.image(device="mixed", output_type=RGB) → GPU, HWC uint8
RandomCrop(32, padding=4)	fn.paste(ratio=1.25, fill_value=0) (가운데 배치, 40×40) + CMN crop=(32,32), 무작위 crop_pos_x/y
RandomHorizontalFlip()	mirror = fn.random.coin_flip(0.5)를 CMN에 전달
ToTensor + Normalize	CMN mean/std (×255), dtype=FLOAT, output_layout="CHW"
Cutout(16) (정규화 후)	제공된 fn_dali_cutout (fn.erase, 0으로 채움)

CMN은 fn.crop_mirror_normalize다. seed는 12345 + shard_id로 두어 rank마다 augmentation 난수가 다르게 했다. 클래스 폴더는 알파벳순으로 정렬되는데, 이는 torchvision의 CIFAR-10 라벨 순서와 같다. 알려진 차이는 두 가지다. 제공된 cutout은 사각형을 항상 이미지 안에 두고, PNG는 JPEG와 달리 mixed 디코더를 써도 CPU에서 디코딩된다.

P7 (DALI 로더). train/test마다 batch batch // world_size, device_id = 현재 GPU, shard_id = rank로 CifarPipeline을 만들고 DALIWrapper(DALIGenericIterator(pipe, ["data", "label"], reader_name="Reader", last_batch_policy=PARTIAL, auto_reset=True))로 감쌌다. PARTIAL은

drop_last=False와 같은 동작이다. DALIWrapper가 StopIteration을 그대로 올리므로 auto_reset이 필요하다.

동작 검증. 2 GPU(각 98 iteration)에서 DDP와 DDP+DALI의 1 epoch train loss가 1.7676과 1.7675로 같고, test 정확도도 48.5%와 47.6%로 비슷하다. 따라서 DALI 파이프라인은 torchvision 파이프라인과 같은 동작을 한다.

2 실험 환경과 분석 방법

주 데이터는 TA가 제공한 로그(RTX 3090, 세 방식 모두 1/2/4 GPU)다. 하드웨어 GPU 지표(SMs Active, DRAM 대역폭)가 이 로그에만 있기 때문이다. 이 지표를 모으려면 Vast.ai 컨테이너에 없는 권한이 필요하다. 우리 실험(Vast.ai, RTX 3090 2장, GPU 간 P2P 불가, 1/2 GPU)은 같은 경향을 재현하는지 확인하는데 썼다(표 2). 각 .nsys-rep는 SQLite로 내보낸 뒤 report/analyze.py로 요약했다. 모든 수치는 NVTX Epoch 0 구간 안에서 재고 rank 평균을 냈다. iteration 하나(epoch당 98개)는 NVTX 행으로 다음과 같이 나뉜다.

- **batch 구간**: NVTX Batch i / Train batch 구간. forward와 backward(+upload, loss)로 나뉜다.
- **batch 밖**: iteration 시간에서 batch 구간을 뺀 나머지. next(loader)와 지표 all-reduce/.item()이 들어가며, 후자는 GPU에 쌓인 작업이 끝나기를 기다린다.

NVTX 구간은 CPU 시간을 쟁다. 그래서 GPU별로 NCCL이 아닌 커널의 실행 구간을 합친 *compute busy*도 함께 보고한다. NCCL 커널은 다른 rank를 기다리는 동안에도 실행 중으로 남아 있어서 뻗다.

3 결과

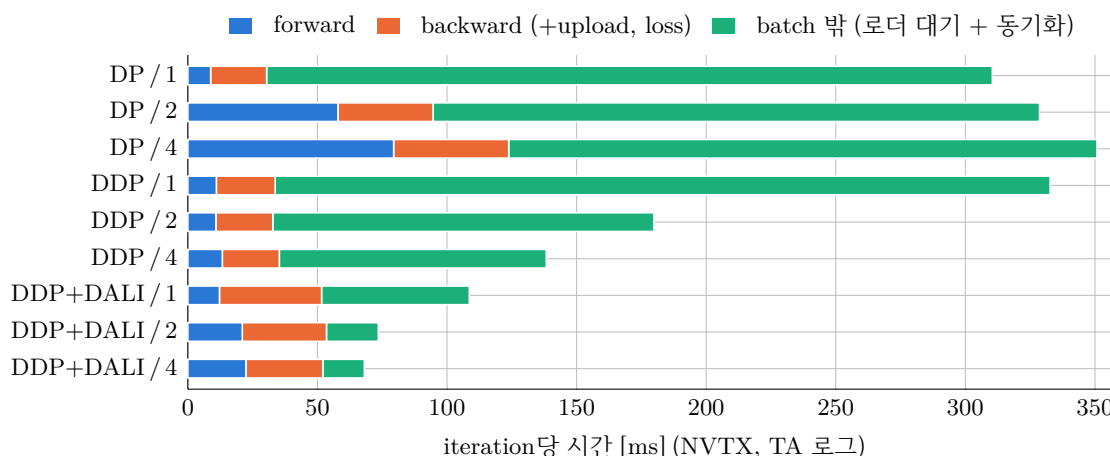


그림 1: NVTX 행으로 나눈 iteration당 시간 (방식 / GPU 수). 수치는 표 1에 있다.

표 1: NVTX 분해와 GPU 지표 (TA 로그, GPU 평균). fwd/bwd/outside = forward/backward/batch 밖, busy = epoch 동안 연산 커널이 실행된 비율.

Mode	GPU	NVTX [ms / iter]					GPU [%]			per GPU	
		epoch [s]	iter	fwd	bwd	outside	SMs act.	DRAM R/W	busy	kernel ms/it	NCCL [s]
DP	1	30.4	310	8.8	20.9	280	28.0	9.1 / 5.3	31.1	96.4	–
	2	32.2	328	57.9	35.9	234	14.1	4.4 / 2.6	15.9	52.3	0.98
	4	34.4	351	79.4	43.1	227	6.8	2.2 / 1.3	7.8	27.5	2.50
DDP	1	32.6	333	11.0	22.1	299	26.3	8.5 / 5.0	29.2	97.0	–
	2	17.7	180	10.8	21.4	147	24.8	8.0 / 4.8	28.3	51.1	2.18
	4	13.6	138	13.3	21.3	103	17.0	5.5 / 3.5	22.4	31.0	4.01
DDP+DALI	1	10.6	109	12.2	38.7	57	82.0	25.7 / 15.1	91.1	99.0	–
	2	7.2	74	21.0	31.6	20	61.2	19.3 / 11.6	70.2	51.7	1.81
	4	6.7	69	22.4	28.9	16	34.1	11.0 / 7.0	45.9	31.5	3.44

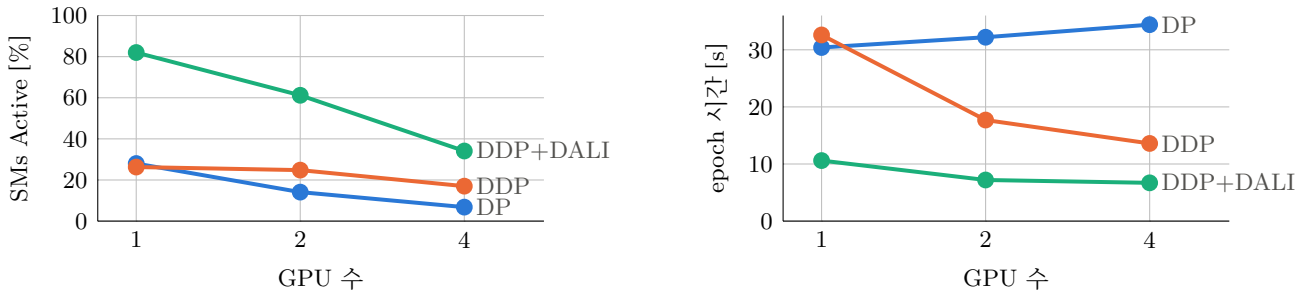


그림 2: GPU 수에 따른 GPU 사용률(SMs Active, 왼쪽)과 epoch 시간(오른쪽) (TA 로그, 수치는 표 1).

표 2: 우리 Vast.ai 실행 (epoch 동안 nvidia-smi를 200 ms 간격으로 기록; mem. = GPU당 최대 할당량 / nvidia-smi 최대값).

Mode	GPU	epoch [s]	iter [ms]	outside [ms]	busy [%]	smi GPU/mem [%]	mem. [MiB]	H→D [MiB]
DP	1	27.8	284	259	37.9	40 / 21	3544 / 3854	586
DP	2	33.4	341	261	14.9	44 / 8	1939 / 2312	881
DDP	1	27.7	282	252	38.2	37 / 18	3662 / 4022	586
DDP	2	18.2	186	158	38.4	48 / 15	1957 / 2378	586
DDP+DALI	1	11.7	119	90	93.9	92 / 45	4086 / 4552	158
DDP+DALI	2	9.7	99	65	74.8	90 / 28	2389 / 2748	158

4 분석: GPU/메모리 사용률이 어떻게, 왜 달라지는가

기준점 (1 GPU). torchvision 로더(num_workers=0)를 쓰면 310–333 ms인 iteration 중 280–300 ms가 batch 밖 시간이다. 우리 로그에서 로더 자체의 Data 타이머가 iteration당 0.17s여서, 이 시간이 데이터 로딩임을 확인했다. 이미지 512장의 PNG 디코딩과 augmentation이 CPU 스레드 하나에서 돌고, GPU가 iteration당 처리할 커널은 약 97 ms뿐이다. 그래서 GPU는 시간의 약 70%를 놓고 있다 (SMs Active 26–28%, DRAM 읽기 9%). DP와 DDP 모두 이 CPU 병목 상태에서 출발한다.

DP는 GPU를 늘릴수록 느려진다. GPU당 커널 시간은 절반씩 줄지만(96→52→28 ms), 사용률은 대략 1/N로 떨어지고 (SMs Active 28→14→7%, DRAM 읽기 9→4→2%) epoch은 오히려 길어진다 (30.4→34.4s). 이유는 두 가지다. (i) 프로세스 하나가 여전히 iteration마다 512장을 모두 로딩해서 batch 밖 시간이 227–280 ms로 줄지 않고, 늘어난 GPU는 더 오래 기다릴 뿐이다. (ii) batch 안에서 DP가 하는 일이 늘어난다. forward가 8.8 ms에서 79.4 ms로 늘어나는데, 매 iteration 입력을 나누고, 파라미터를 복제하고 (ncclBroadcast), GIL 아래에서 Python 스레드로 복제본을 돌리고, 출력을 GPU 0에 모으기 때문이다. backward도 gradient ncclReduce 때문에 21 ms에서 43 ms로 늘어난다. 입력은 PCIe를 두 번 지난다. H→D는 586 MiB에서 881/1029 MiB로, D→H는 0에서 295/442 MiB로 느는데, 이는 정확히 batch의 $(N-1)/N$ 이다. GPU 0이 입력을 호스트 메모리를 거쳐 나눠 주기 때문이다. TA가 경고한 통신 overhead가 바로 이것으로, GPU당 유효 연산은 줄어드는데 이 비용은 N 과 함께 커진다. 복제본마다 512/N장의 activation만 가지므로 GPU당 메모리도 절반이 된다(할당량 3544→1939 MiB).

DDP는 프로세스 수만큼 빨라진다. DDP는 GPU마다 프로세스를 하나씩 두므로, 각자 자기 몫 512/N장만 병렬로 로딩한다. batch 밖 시간은 299→147→103 ms로, epoch은 32.6→17.7→13.6s로 줄어 4 GPU에서 2.4배 빨라진다. batch 구간은 약 33 ms로 일정하다. 파라미터는 처음에 한 번만 맞추고(Sync model parameters), step마다 작은 BatchNorm 버퍼만 broadcast한다(ncclBroadcast는 모두 forward 안에 있다). DP처럼 입력을 나누거나 모델을 복제하지 않는다. gradient는 bucket 2–3개로 나눠 backward와 겹쳐서 all-reduce한다. 2 GPU에서 ncclAllReduce 1368회 중 544회가 backward 안에 있고, 나머지 대부분은 batch 밖에서 지표를 합산하는 reduce_tensor다. 그 결과 2 GPU까지는 GPU당 사용률이 거의 유지된다(SMs 26→25%, DRAM 8.5→8.0%). 4 GPU에서는 17%로 떨어진다. GPU당 연산(31 ms)이 로더 시간보다 빨리 줄고, NCCL 커널(GPU당 4.0s)이 가장 느린 rank를 기다리는 시간이 늘기 때문이다. bucket 복사는 GPU당 epoch마다 4.2 GiB의 D→D 전송으로 나타난다. GPU당 최대 메모리는 DP와 같고 (2 GPU에서 1957 MiB) rank끼리 균등하다.

DALI는 로더 병목을 없앤다. DALI는 batch를 미리 준비해 두고 crop/flip/normalize/cutout을 GPU에서 학습과 겹쳐 실행한다. 1 GPU에서 batch 밖 시간은 299 ms에서 57 ms로, epoch은 32.6s에서 10.6s로 줄어 3.1배 빨라진다. GPU가 바빠지고(SMs Active 82%, compute busy 91%, nvidia-smi 92%) 메모리 대역폭 사용도 약 3배가 된다(DRAM 읽기 26%, nvidia-smi 메모리 사용률 45% 대 18%). 남은 batch 밖

시간의 대부분은 이제 데이터가 아니라 `.item()`이 쌓인 커널을 기다리는 시간이다. 정규화된 float32 텐서 대신 uint8 이미지를 올리므로 H→D 전송량이 3.7배 줄어든다 (586→158-172 MiB). 비용도 있다. 파이프라인을 만드느라 `Set data loader`가 1.0s에서 3.3s로 늘고, DALI 버퍼 때문에 GPU 메모리를 약 430 MiB 더 쓴다(4086 대 3662 MiB). GPU를 늘리면 이번에는 다른 한계에 걸린다. 4 GPU에서 GPU당 커널 시간은 31ms까지 줄지만 CPU 쪽 batch 구간은 약 52ms로 그대로여서, batch 128짜리 ResNet-18의 커널을 CPU가 넣는 시간이 전체를 좌우한다. SMs Active는 61%(2 GPU), 34%(4 GPU)로 떨어지고 epoch은 10.6→7.2→6.7s로 조금만 줄어든다.

참고. (1) `nvidia-smi`의 GPU 사용률은 다른 rank를 기다리는 NCCL 커널까지 포함해 실행 중인 커널이 하나라도 있으면 사용 중으로 센다. 우리 DP 2 GPU 실행에서 44%로 나오지만 연산 커널은 15%만 실행됐다. 그래서 SMs Active와 compute busy를 썼다. (2) TA 로그의 *GPU Active*는 일부 GPU에서 SMs Active가 다른 GPU와 같은데도 100%로 나와서 쓰지 않았다. (3) 우리 인스턴스는 GPU 간 P2P를 쓸 수 없고, DDP 2 GPU의 GPU당 커널 시간이 TA보다 길다(71 대 51 ms). 그래도 경향과 순서(DP < DDP < DDP+DALI)는 TA 로그와 같다.

결론. DP는 프로세스 하나짜리 데이터 로더와 매 iteration의 입력 분배·모델 복제·출력 수집에 묶여 있어서, GPU를 늘리면 사용률이 떨어지고 학습이 느려진다. DDP는 로딩을 프로세스마다 나누고 gradient all-reduce를 backward와 겹쳐, 로딩이 병목인 동안에는 GPU 수에 가깝게 빨라진다. DALI는 augmentation을 GPU로 옮기고 비동기로 미리 준비해서, 1 GPU에서 GPU 사용률과 메모리 대역폭 사용률을 80-90% 수준으로 끌어올린다. 그다음 병목은 GPU당 batch가 작을 때 CPU가 커널을 넣는 overhead다.

5 최종 loss와 정확도

1 epoch 학습 후 train loss는 모든 실행에서 1.74-1.81로 비슷하지만, test 정확도는 46.6%에서 55.2%까지 차이가 난다 (1/2 GPU 기준 DP 55.2/49.2%, DDP 48.8/50.5%, DDP+DALI 46.6/47.7%). 기댓값으로는 세 방식이 같은 SGD를 실행한다. 모델, optimizer, augmentation, 전체 batch 512가 같고, DDP가 256장 평균 두 개를 all-reduce한 값은 DP의 512장 평균과 같다. 차이는 대부분 실행마다 생기는 편차다. `--seed`가 실제로 적용되지 않아 초기 가중치, 데이터 순서, augmentation이 매번 다르고, 학습이 lr 0.04의 warm-up epoch 하나뿐이다. 실제로 DDP 2 GPU를 다시 돌리면 50.5%와 48.5%가 나왔다. 작은 구조적 차이로는 GPU당 BatchNorm 통계가 512장이 아닌 256장에서 계산되는 점(SyncBN 미사용), sampler마다 다른 데이터 순서, DALI의 cutout 위치 분포가 있다. DALI reader가 클래스 순으로 정렬된 파일 목록을 버퍼 안에서만 섞을 가능성도 있다(확인하지 못함; DALI는 세 번 모두 1-2%p 낮았다). DP 로그의 loss가 약 500배 작게 보이는 것은 DP/train.py가 평균 loss를 batch 크기로 한 번 더 나누기 때문이다.